

数据结构与算法课程设计

回溯算法：47.全排列II

1、程序代码

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> path;
    vector<vector<int>> result;
    void backtracking(vector<int>& nums, vector<int>& flag) {
        if (path.size() == nums.size()) {
            result.push_back(path);
            return;
        }
        for (int i = 0; i <= nums.size() - 1; i++) {
            if (flag[i] || (i > 0 && nums[i] == nums[i - 1] && flag[i - 1] == 0))
                continue;
            flag[i] = 1;
            path.push_back(nums[i]);
            backtracking(nums, flag);
            path.pop_back();
            flag[i] = 0;
        }
    }
    vector<vector<int>> permuteUnique(vector<int>& nums) {
        vector<int> flag(nums.size(), 0);
        sort(nums.begin(), nums.end());
        backtracking(nums, flag);
        return result;
    }
};

int main() {
    int t;
    cin >> t;
    while (t--) {
        Solution sol;
        vector<int> nums;
        int n;
        cin >> n;
        for (int i = 0; i < n; ++i) {
            int num;
            cin >> num;
            nums.push_back(num);
        }
        vector<vector<int>> permutations = sol.permuteUnique(nums);
        cout << "全排列结果如下: " << endl;
        for (const auto& perm : permutations) {
            for (int num : perm) {
                cout << num << " ";
            }
            cout << endl;
        }
    }
    return 0;
}
```

2、运行结果

```
2
3
1 1 2
全排列结果如下：
1 1 2
1 2 1
2 1 1
3
1 2 3
全排列结果如下：
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1

Process returned 0 (0x0)   execution time : 3.161 s
Press any key to continue.
```

3、实习总结与体会

学习到了回溯算法的思想。

(1) 回溯算法：回溯法也可以叫做回溯搜索法，它是一种搜索的方式。回溯是递归的副产品，只要有递归就会有回溯。

- 回溯算法可以解决组合、切割、子集、排列、棋盘问题
- 回溯算法模板：

```
void backtracking(参数) {
    if (终止条件) {
        存放结果;
        return;
    }

    for (选择：本层集合中元素（树中节点孩子的数量就是集合的大小）) {
        处理节点;
        backtracking(路径，选择列表); // 递归
        回溯，撤销处理结果
    }
}
```

(2) 该题与上一题的区别是，这道题要记得数组内元素可能有重复。关键解决代码：

```
if (flag[i] || (i > 0 && nums[i] == nums[i - 1] && flag[i - 1] == 0)) continue;
```